

LAPORAN TUGAS KECIL III STRATEGI ALGORITMA

Ice Sliding Puzzle Solver

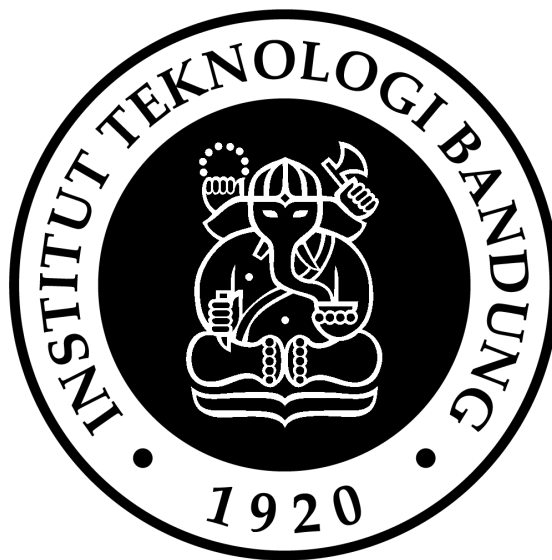
Dipersiapkan oleh:

Aufa Rienaldifaza Ahmad

13524027

Yavie Azka Putra Araly

13524077



**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESHA 10, BANDUNG 40132
2025**

BAB I	3
LANDASAN TEORI	3
1.1. UCS (Uniform Cost Search)	3
1.2. GBFS (Greedy Best-First Search)	5
1.3. A*	7
BAB II	9
ANALISIS ALGORITMA	9
2.1. Definisi Fungsi Estimasi	9
2.2. Argumen Heuristik Admissible	10
2.3. Perbandingan Algoritma	11
BAB III	13
IMPLEMENTASI	13
1. Graph	13
2. UCS	15
2.1. Loop Pencarian Utama dan Pruning	15
3. GBFS	17
4. A*	19
BAB IV	23
PENGUJIAN	23
Referensi	30
Lampiran	31

BAB I

LANDASAN TEORI

1.1. UCS (*Uniform Cost Search*)

Algoritma UCS mengeksplorasi graf dengan memprioritaskan simpul (atau state permainan) yang memiliki total biaya lintasan atau $g(n)$ paling rendah dari titik awal. Algoritma ini menjamin penemuan jalur dengan biaya pergerakan (akumulasi cost traversal seluruh tile yang dilalui saat meluncur) paling minimum.

Pada algoritma ini, nilai $g(n)$ yang digunakan adalah total dari seluruh cost tile yang sudah dilalui untuk mencapai simpul n .

Langkah-langkah implementasi UCS:

1. Struktur data `priority_queue` (sebagai min-heap) disiapkan untuk menyimpan objek `UCSNode` yang diurutkan berdasarkan $gCost$ ($g(n)$) terkecil. Tabel hash `bestCost` digunakan untuk mencatat rekor biaya terendah untuk mencapai suatu state, sedangkan tabel `parent` dan `parentMove` digunakan untuk merekam jejak jalur.
2. State awal permainan dimasukkan ke dalam `priority_queue` dengan nilai $g(n) = 0$. State ini juga dicatat ke dalam `bestCost` dan `parent`.
3. Program melakukan iterasi selama antrian tidak kosong. Pada setiap iterasi, state dengan $gCost$ paling rendah (disebut `cur`) dikeluarkan dari antrian.
4. Jika $gCost$ dari simpul yang diekstraksi ternyata lebih besar dari biaya yang sudah tercatat di `bestCost` untuk state tersebut, maka simpul ini diabaikan (karena jalur yang lebih optimal sudah ditemukan sebelumnya).
5. Jika `cur` merupakan goal state (karakter telah mencapai titik keluar 'O' dan semua checkpoint angka telah dilewati sesuai urutan), iterasi dihentikan.
6. Jika belum mencapai target, program akan membangkitkan semua kemungkinan state berikutnya dari `cur` (meluncur ke atas, bawah, kiri, atau kanan). Untuk setiap suksesor yang valid:
 - a. Dihitung `newCost`, yakni penjumlahan $gCost$ saat ini dengan biaya pergerakan suksesor tersebut.

- b. Apabila state suksesor belum pernah dieksplorasi (tidak ada di bestCost) atau ditemukan jalur baru dengan newCost yang lebih kecil dari catatan di bestCost, maka bestCost diperbarui, riwayat pergerakan dicatat, dan suksesor tersebut dimasukkan ke dalam priority_queue.
7. Setelah goal state ditemukan, program melakukan penelusuran balik (backtracking) menggunakan tabel parent untuk menghasilkan riwayat pergerakan dari titik awal hingga titik akhir.

Berikut merupakan *pseudocode* implementasi algoritma UCS

```
function UCS(graph):
    PQ = new PriorityQueue() // Min-heap yang diurutkan berdasarkan gCost
    bestCost = new Map()    // Menyimpan cost minimum untuk mencapai suatu
state
    parent = new Map()      // Menyimpan state parent untuk rekonstruksi
jalur
    parentMove = new Map()  // Menyimpan jenis gerakan dari parent ke state

    // Kondisi awal
    initState = graph.getInitialState()
    PQ.push( (initState, 0) )
    bestCost[initState] = 0
    parent[initState] = initState
    parentMove[initState] = 'S' // S merepresentasikan Start

    while PQ is not empty:
        curState, curCost = PQ.pop()

        // Jika ditemukan state di queue dengan cost yang lebih buruk
        // dari cost terbaik yang sudah ditemukan sebelumnya, abaikan.
        if curCost > bestCost[curState]:
            continue

        // Cek kondisi berhenti
        if graph.isGoal(curState):
            return reconstructPath(parent, parentMove, initState, curState)

        // Ekspansi state suksesor (gerakan Valid: R, L, U, D)
        for each succ in graph.getSuccessors(curState):
            newCost = curCost + succ.cost

            // Jika state belum dieksplorasi atau ditemuka rute yang lebih
murah
            if succ.state not in bestCost OR newCost < bestCost[succ.state]:
                bestCost[succ.state] = newCost
                parent[succ.state] = curState
                parentMove[succ.state] = succ.move
                PQ.push( (succ.state, newCost) )
```

```
return NO_SOLUTION
```

1.2. GBFS (*Greedy Best-First Search*)

Algoritma GBFS mengabaikan biaya yang sudah dikeluarkan sejauh ini dan hanya berfokus pada estimasi biaya dari posisi saat ini menuju tujuan yang direpresentasikan dengan fungsi heuristik $h(n)$.

Nilai heuristik dihitung berdasarkan Manhattan Distance antara posisi karakter saat ini dengan target selanjutnya. Jika karakter belum melewati semua checkpoint angka, targetnya adalah koordinat checkpoint angka terdekat sesuai urutan. Jika semua checkpoint sudah terlewati, targetnya adalah titik keluar 'O'. Berikut merupakan langkah-langkah implementasi GBFS:

1. Struktur data `priority_queue` disiapkan untuk menyimpan objek `GBFSNode` yang diurutkan murni berdasarkan nilai heuristik $hCost$ terkecil. Tabel hash `parent` dan `parentMove` disiapkan untuk melacak state yang sudah dikunjungi dan jalur pergerakannya.
2. Kondisi Awal: State awal dievaluasi nilai $h(n)$ -nya terhadap target pertama, kemudian dimasukkan ke dalam antrian. State ini ditandai sudah dikunjungi di dalam tabel `parent`.
3. Ekstraksi Simpul: Selama antrian tidak kosong, state dengan estimasi $hCost$ terendah diekstraksi dari antrian untuk diperiksa.
4. Pemeriksaan Solusi: Jika state yang diekstraksi adalah goal state (mencapai titik akhir 'O' setelah mengumpulkan semua checkpoint berurutan), iterasi dihentikan.
5. Ekspansi Suksesor: Program membangkitkan semua pergerakan yang mungkin dari state saat ini. Untuk setiap state suksesor: Diperiksa apakah state tersebut belum pernah dieksplorasi (belum tercatat di tabel `parent`). Jika belum pernah dikunjungi, state tersebut didaftarkan ke tabel `parent`, dihitung nilai $h(n)$ terbarunya, lalu dimasukkan ke dalam `priority_queue` untuk ekspansi lebih lanjut. (Catatan: Karena GBFS bersifat greedy, ia tidak akan memperbarui node jika ditemukan jalur lain ke state yang sama).

6. Rekonstruksi Jalur: Serupa dengan UCS, jika solusi ditemukan, program akan menyusun balik urutan state untuk mendapatkan langkah pathfinding secara utuh menggunakan tabel parentMove.

Berikut merupakan *pseudocode* implementasi GBFS:

```
function calculateManhattan(state, graph):
    checkpoints = graph.getCheckpointPositions()
    totalCp = graph.getCheckpointCount()
    target = graph.getGoalPos() // Default target adalah titik keluar '0'

    // Cari checkpoint angka terdekat yang belum dikunjungi sesuai urutan
    for i from 0 to totalCp - 1:
        // Pengecekan menggunakan bitwise AND berdasarkan mask state
        if not (state.visitedCheckpoints HAS_BIT i):
            target = checkpoints[i]
            break

    // Kembalikan jarak Manhattan dari posisi saat ini ke target
    return abs(state.pos.row - target.row) + abs(state.pos.col - target.col)

function GBFS(graph):
    // Inisialisasi struktur data
    PQ = new PriorityQueue() // Min-heap yang diurutkan berdasarkan hCost
    parent = new Map() // Digunakan untuk tracking visited state dan
    jalur
    parentMove = new Map()

    // Kondisi awal
    initState = graph.getInitialState()
    initH = calculateManhattan(initState, graph)

    PQ.push( (initState, initH) )
    parent[initState] = initState
    parentMove[initState] = 'S'

    while PQ is not empty:
        curState, curH = PQ.pop()

        // Cek kondisi berhenti
        if graph.isGoal(curState):
            return reconstructPath(parent, parentMove, initState, curState)

        // Ekspansi state suksesor
        for each succ in graph.getSuccessors(curState):
            // Karena sifatnya Greedy, kita hanya mempedulikan state yang
            belum pernah dikunjungi
            if succ.state not in parent:
                parent[succ.state] = curState
                parentMove[succ.state] = succ.move

                hCost = calculateManhattan(succ.state, graph)
                PQ.push( (succ.state, hCost) )
```

```
return NO_SOLUTION
```

1.3. A*

Algoritma A* mempertimbangkan total estimasi biaya yang direpresentasikan oleh fungsi $f(n) = g(n) + h(n)$, $g(n)$ merepresentasikan biaya kumulatif yang telah dikeluarkan dari state awal hingga state saat ini, dan $h(n)$ merepresentasikan estimasi biaya dari posisi saat ini menuju tujuan yang dihitung menggunakan fungsi heuristik. Nilai heuristik dihitung berdasarkan Manhattan Distance terurut dari posisi karakter saat ini ke setiap checkpoint yang belum dikunjungi secara berurutan, lalu ke titik keluar 'O', seluruhnya dikalikan dengan bobot tile minimum (`minWeight`) untuk menjamin sifat admissible. Berikut merupakan langkah-langkah implementasi A*:

1. Struktur data `priority_queue` digunakan untuk menyimpan objek `Node` yang diurutkan berdasarkan nilai $f(n)$ terkecil. Selain itu, disiapkan `closedList` untuk mencatat state yang sudah diekspansi, kemudian biaya kumulatif $g(n)$ disimpan berdasarkan nilai terbaik yang diketahui, dan `map` untuk menyimpan `cameFromMove` dan `cameFromState` untuk melacak state induk dan arah pergerakan pada jalur pencarian.
2. State awal dievaluasi nilai $h(n)$ -nya terhadap seluruh target terurut (checkpoint yang belum dikunjungi lalu goal), kemudian dimasukkan ke dalam *closed list* dengan nilai $g = 0$ dan $f = g + h = h$. State ini dicatat dalam `gScore` dengan nilai 0, `cameFrom` dengan merujuk ke dirinya sendiri, dan `cameFromMove` dengan nilai 'S' sebagai state awal.
3. Selama antrian tidak kosong, state dengan nilai $f(n)$ terendah diekstraksi dari antrian untuk diperiksa. Penghitung iterasi ditambah setiap kali ekstraksi dilakukan.
4. State yang baru diekstraksi diperiksa apakah merupakan *stale node*, yaitu jika nilai g pada node tersebut lebih besar dari nilai g yang tercatat di `gScore`. Jika ya, node tersebut dilewati karena sudah ditemukan jalur yang lebih baik sebelumnya. Selain itu, jika state sudah terdapat di `closedList` (sudah pernah diekspansi), node

tersebut juga dilewati. Jika kedua pemeriksaan lolos, state ditandai sebagai telah diekspansi ke dalam closedList.

5. Pemeriksaan Solusi: Jika state yang diekstraksi adalah goal state (mencapai titik akhir 'O' setelah mengumpulkan semua checkpoint berurutan), iterasi dihentikan.
6. Ekspansi Suksesor: Program membangkitkan semua pergerakan yang mungkin dari state saat ini. Untuk setiap state suksesor: Jika suksesor sudah ada di closedList, ia dilewati. Jika belum, dihitung nilai $possibleG = cur.g + succ.cost$ (biaya kumulatif baru menuju suksesor melalui jalur saat ini). Jika suksesor belum pernah dicatat di gScore atau possibleG lebih kecil dari gScore yang sudah ada, maka gScore diperbarui ke possibleG, cameFrom dan cameFromMove diperbarui ke state saat ini, dan nilai f baru dihitung sebagai $possibleG + heuristic(succ.state, graph)$. Node kemudian dimasukkan kembali ke dalam antrean prioritas.
7. Jika solusi ditemukan, program menyusun balik urutan state dari goalState ke initState menggunakan tabel cameFrom dan cameFromMove. Hasil penelusuran balik dibalik urutannya, lalu snapshot awal serta setiap langkah pergerakan dicatat ke dalam log solusi.

BAB II

ANALISIS ALGORITMA

2.1. Definisi Fungsi Estimasi

2.1.1. UCS

g(n): Merupakan biaya nyata (actual cost) yang dikeluarkan untuk mencapai state n dari state awal. Berdasarkan spesifikasi, biaya pergerakan dihitung dari akumulasi cost seluruh tile yang dilalui selama satu gerakan meluncur (sliding), kecuali tile posisi awal. Dalam kode ucs.cpp, nilai ini disimpan dalam variabel gCost dan diakumulasikan setiap kali suksesor baru ditemukan.

2.1.2. GBFS

h(n): Merupakan nilai heuristik atau estimasi biaya dari state n menuju target terdekat (baik itu checkpoint angka berikutnya atau titik keluar 'O'). Dalam kode gbfs.cpp, h(n) dihitung menggunakan Manhattan Distance yang disesuaikan dengan urutan checkpoint (checkpoint-aware).

2.1.3. A*

g(n): merepresentasikan total biaya (cost) aktual yang telah dikeluarkan untuk mencapai state n dari titik awal permainan. Nilai ini didapatkan dari akumulasi cost traversal setiap tile (berdasarkan matriks input bagian kedua) yang dilewati karakter selama meluncur. Pada implementasi program, nilai g(n) direkam di dalam struktur tabel gScore, yang akan selalu diperbarui (melalui proses relaksasi) apabila algoritma menemukan jalur alternatif yang lebih murah menuju state yang sama.

h(n): Fungsi h(n) bertugas memberikan estimasi biaya paling minimum dari state n menuju ke titik keluar ('O'), dengan tetap mempertimbangkan kewajiban melewati checkpoint angka secara berurutan. Berdasarkan implementasi fungsi heuristic yang dirancang, h(n) dihitung dengan mekanisme berikut:

- Algoritma menghitung total Manhattan Distance secara berantai, yaitu dari posisi karakter saat ini menuju ke checkpoint angka pertama yang belum dikunjungi, lalu berlanjut ke checkpoint kedua, dan seterusnya secara berurutan, hingga akhirnya mengarah ke koordinat tujuan akhir.

- Total jarak kumulatif tersebut kemudian dikalikan dengan bobot tile paling minimum yang ada pada peta (diambil menggunakan `graph.getMinWeight()`). Langkah ini sangat krusial karena berfungsi mengonversi satuan metrik "jarak langkah" menjadi "estimasi biaya" yang sepadan dengan besaran $g(n)$, sekaligus menjaga keabsahan (admissibility) fungsi heuristik agar tidak pernah melakukan overestimate terhadap biaya aslinya.

f(n): adalah penjumlahan antara $g(n)$ dan $h(n)$. Nilai ini melambangkan estimasi total biaya dari rute paling optimal (termurah) mulai dari titik awal hingga ke titik akhir dengan asumsi jalur tersebut melewati state n . Dalam struktur data, nilai $f(n)$ ini dijadikan dasar perbandingan (ACompare) pada antrean prioritas (priority_queue). Dengan demikian, node atau state dengan nilai $f(n)$ terkecil (estimasi total biaya termurah) akan selalu diprioritaskan untuk diekspansi lebih dahulu.

2.2. Argumen Heuristik Admissible

Fungsi heuristik A^* pada implementasi ini dihitung sebagai $minWeight \times \sum_{i=0}^n manhattan_i$ dengan n merupakan jumlah checkpoint

Manhattan dari posisi karakter saat ini ke checkpoint pertama yang belum dikunjungi, lalu berurutan ke checkpoint berikutnya, hingga akhirnya ke posisi goal. Manhattan Distance antara dua posisi pada grid merepresentasikan jumlah langkah grid minimum untuk berpindah antar posisi tersebut, sehingga jumlah tile yang dilalui pada jalur aktual mana pun selalu $\geq$ Manhattan Distance. Selain itu, karena setiap tile memiliki bobot $\geq minWeight$, maka biaya minimal untuk melintasi sejumlah tile sama dengan jumlah tile dikalikan $minWeight$. Perkalian ini menjamin bahwa setiap satuan jarak diberi nilai sekecil mungkin yang mungkin terjadi di papan.

Karena aturan permainan mengharuskan checkpoint dikunjungi secara sekuensial ($0 \rightarrow 1 \rightarrow 2 \rightarrow \dots$), setiap jalur solusi valid wajib melewati seluruh checkpoint yang belum dikunjungi sesuai urutan sebelum mencapai goal, sehingga

penjumlahan Manhattan Distance terurut tidak mengabaikan destinasi yang harus dilalui. Di sisi lain, jalur aktual mungkin memerlukan langkah tambahan akibat menghindari dinding/lava atau sifat sliding yang memaksa karakter meluncur melewati target, sehingga jumlah tile aktual antara dua target selalu $\geq$ Manhattan Distance keduanya. Dengan demikian, $h(n) = \text{minWeight} \times \Sigma \text{Manhattan terurut}$ merupakan batas bawah dari biaya optimal menuju goal, dan karena $h(n)$ tidak pernah melebihi biaya aktual terpendek, fungsi heuristik ini terbukti *admissible*.

2.3. Perbandingan Algoritma

Karakteristik	UCS	GBFS	A*
Optimalitas	Menjamin solusi dengan biaya termurah (<i>optimal</i>)	Tidak menjamin solusi optimal, hanya fokus pada arah tujuan.	Menjamin solusi optimal (karena heuristik <i>admissible</i>).
Kelengkapan	<i>Complete</i> (pasti menemukan solusi jika ada).	<i>Complete</i> pada ruang keadaan terbatas.	<i>Complete</i> .
Efisiensi	Cenderung lambat karena mengekskansi semua <i>node</i> dengan <i>cost</i> rendah ke segala arah (seperti <i>breadth-first</i>).	Sangat cepat karena langsung menuju target, namun bisa terjebak di jalur yang salah jika ada banyak rintangan.	Paling efisien karena menggabungkan akurasi biaya UCS dengan panduan arah dari GBFS.
Relevansi Kasus	Cocok jika tujuan utama adalah meminimalkan total <i>cost traversal tile</i> .	Cocok untuk menemukan solusi cepat pada <i>map</i> yang sederhana tanpa mempedulikan <i>cost</i> .	Pilihan terbaik untuk menemukan rute paling hemat biaya dengan waktu eksekusi yang optimal.

BAB III

IMPLEMENTASI

1. Graph

1.1. Pembangkitan Suksesor dan Mekanika Peluncuran (getSuccessors)

Fungsi ini adalah inti dari mekanika sliding puzzle. Algoritma tidak membangkitkan pergerakan per satu petak (tile), melainkan mensimulasikan pergerakan meluncur hingga karakter berhenti secara otomatis akibat menabrak dinding batas papan atau rintangan batu ('X').

Selama karakter meluncur dalam satu arah (while (true)), algoritma melakukan tugas-tugas berikut:

1. Akumulasi Cost: Menambahkan bobot cost dari tile yang sedang dilewati ke dalam totalCost.
2. Validasi Lava ('L'): Jika karakter melewati lava, status gameOver diaktifkan dan rute tersebut dianggap tidak valid.
3. Validasi Urutan Checkpoint: Jika karakter menginjak angka, program menggunakan operasi bitwise (visitedBits) untuk mengecek apakah angka tersebut adalah checkpoint yang tepat untuk diambil saat ini. Jika urutannya salah, langkah tersebut memicu gameOver.

```
std::vector<Successor> Graph::getSuccessors(const State& state) const {  
    std::vector<Successor> successors;  
  
    // Evaluasi 4 arah: Atas, Bawah, Kiri, Kanan  
    for (int d = 0; d < 4; d++) {  
        // ... (inisialisasi koordinat awal gerakan)  
        if (!isInBounds(nr, nc) || grid_[nr][nc] == 'X') continue;  
  
        int totalCost = 0;  
        int visitedBits = state.visitedCheckpoints;  
        bool gameOver = false;
```

```

// Simulasi meluncur hingga mentok
while (true) {
    nr = cr + DR[d];
    nc = cc + DC[d];

    if (!isInBounds(nr, nc) || grid_[nr][nc] == 'X') break;

    cr = nr;
    cc = nc;
    totalCost += weights_[cr][cc]; // Akumulasi cost

    if (grid_[cr][cc] == 'L') { // Terbakar Lava
        gameOver = true;
        break;
    }

    // Validasi Checkpoint
    int cpIdx = getCheckpointIndex(cr, cc);
    if (cpIdx != -1) {
        int nextReq = getNextRequiredCheckpoint(visitedBits);
        if (cpIdx == nextReq) {
            visitedBits |= (1 << cpIdx); // Tandai sudah dikunjungi
            (bitwise OR)
        } else if (visitedBits & (1 << cpIdx)) {
            // Checkpoint sudah pernah diambil sebelumnya, anggap
            sebagai tile normal
        } else {
            gameOver = true; // Mengambil checkpoint dengan urutan
            salah
            break;
        }
    }
}

if (gameOver) continue;
if (cr == state.pos.row && cc == state.pos.col) continue; // Tidak ada

```

```
pergerakan
```

```
        successors.emplace_back(Successor{{{cr, cc}, visitedBits}, totalCost,  
MOVES[d]}));  
    }  
    return successors;  
}
```

1.2. Kondisi Kemenangan (isGoal)

Algoritma memeriksa apakah sebuah state dapat diklaim sebagai solusi akhir. Sebuah state dinyatakan menang bukan hanya jika koordinat posisi karakter (state.pos) sama persis dengan koordinat titik tujuan ('O'), melainkan juga harus lolos verifikasi bitwise di mana semua checkpoint yang ada di peta(totalCheckpoints_) telah berhasil dikumpulkan secara berurutan.

2. UCS

2.1. Loop Pencarian Utama dan *Pruning*

Pencarian berlangsung selama antrian tidak kosong dan goal belum tercapai. Setiap iterasinya, algoritma mengekstraksi node teratas (biaya termurah) dan menambahkan counter iterasi.

Terdapat mekanisme pruning penting di bagian ini: jika gCost dari node yang diekstraksi ternyata lebih besar dari rekor bestCost yang tersimpan, algoritma akan melakukan continue untuk mengabaikan node usang (stale) tersebut dan menghemat waktu komputasi.

```
while (!pq.empty() && !found) {  
    UCSNode cur = pq.top();  
    pq.pop();  
    iterations++;  
  
    // Pruning: lewati state jika sudah ditemukan rute yang lebih murah  
    sebelumnya  
    if (cur.gCost > bestCost[cur.state]) {
```

```

        continue;
    }

    if (graph.isGoal(cur.state)) {
        goalState = cur.state;
        found = true;
        break;
    }
    // ...
}

```

2.2. Ekspansi dan Relaksasi

Ketika sebuah state belum mencapai tujuan, algoritma membangkitkan daftar pergerakan yang valid menggunakan `graph.getSuccessors`.

Untuk setiap kemungkinan gerakan:

1. Dihitung biaya kumulatif baru (`newCost`) dengan menjumlahkan `gCost` saat ini dan `cost` dari gerakan tersebut.
2. Proses relaksasi terjadi jika state suksesor belum pernah dieksplorasi (tidak ada di `bestCost`) atau jika algoritma berhasil menemukan rute baru dengan `newCost` yang lebih rendah dari rekor sebelumnya. Jika syarat ini terpenuhi, maka tabel `bestCost` diperbarui, jejak orang tua (`parent`) dicatat, dan suksesor dilemparkan kembali ke dalam antrian prioritas.

```

for (const auto& succ : graph.getSuccessors(cur.state)) {
    int newCost = cur.gCost + succ.cost;

    auto it = bestCost.find(succ.state);
    // Masukkan ke queue hanya jika state baru atau ditemukan cost yang lebih
    kecil
    if (it == bestCost.end() || newCost < it->second) {
        bestCost[succ.state] = newCost;
        parent[succ.state] = cur.state;
        parentMove[succ.state] = succ.move;
    }
}

```



```
        pq.push({succ.state, newCost});  
    }  
}
```

2.3. Rekonstruksi Jalur

Apabila solusi berhasil ditemukan, algoritma melakukan penelusuran balik (backtracking) mulai dari goalState merunut ke belakang dengan membaca peta parent dan parentMove hingga mencapai initState. Jalur yang terbentuk kemudian dibalik (std::reverse) agar urutannya menjadi kronologis dari titik awal. Terakhir, sistem menyusun sekumpulan snapshot yang berisi posisi dan rekam jejak gerakan untuk divisualisasikan kepada pengguna.

3. GBFS

3.1. Fungsi Heuristik

Inti dari kecerdasan GBFS pada program ini terletak pada fungsi manhattan. Fungsi ini dirancang agar menyadari keberadaan checkpoint angka (checkpoint-aware) sehingga karakter tidak langsung meluncur ke titik akhir ('O') jika masih ada angka yang belum diambil.

- Algoritma membaca status checkpoint yang sudah dikunjungi menggunakan operasi bitwise (state.visitedCheckpoints & (1 << i)).
- Jika ditemukan checkpoint yang belum dikunjungi, maka koordinat checkpoint tersebut ditetapkan sebagai target. Jika semua checkpoint sudah dikumpulkan, maka targetnya adalah goalPos (titik keluar).
- Fungsi kemudian mengembalikan nilai Manhattan Distance (selisih absolut baris dan kolom) antara posisi karakter saat ini dengan target tersebut.

```
static int manhattan(const core::State& state, const core::Graph& graph) {  
    const auto& checkpoints = graph.getCheckpointPositions();  
    int totalCp = graph.getCheckpointCount();
```

```

core::Point target = graph.getGoalPos();
// Mencari checkpoint terdekat sesuai urutan yang belum dikunjungi
for (int i = 0; i < totalCp; i++) {
    if (!(state.visitedCheckpoints & (1 << i))) {
        target = checkpoints[i];
        break;
    }
}

// Mengembalikan nilai Manhattan Distance ke target
return std::abs(state.pos.row - target.row)
       + std::abs(state.pos.col - target.col);
}

```

3.2. Siklus Ekspansi

Selama antrian tidak kosong, algoritma mengekstraksi node teratas (cur). Jika cur adalah goal state, pencarian langsung dihentikan. Jika tidak, program akan membangkitkan semua suksesor yang mungkin.

Karakteristik greedy sangat terlihat pada bagian ini: algoritma hanya mengecek apakah state suksesor belum pernah dieksplorasi (belum ada di dalam tabel parent). Berbeda dengan UCS atau A* yang mengevaluasi dan memperbarui jalur (cost relaxation), GBFS langsung mendaftarkan suksesor baru tersebut, menghitung nilai heuristiknya, dan memasukkannya ke dalam antrian.

```

while (!pq.empty() && !found) {
    GBFSNode cur = pq.top();
    pq.pop();
    iterations++;

    if (graph.isGoal(cur.state)) {
        goalState = cur.state;
        found = true;
        break;
    }
}

```

```

    }

    for (const auto& succ : graph.getSuccessors(cur.state)) {
        // Jika state suksesor belum pernah dikunjungi sama sekali
        if (parent.find(succ.state) == parent.end()) {
            parent[succ.state] = cur.state;
            parentMove[succ.state] = succ.move;
            // Masukkan ke antrean dengan prioritas berdasarkan nilai
            heuristiknya
            pq.push({succ.state, manhattan(succ.state, graph)});
        }
    }
}

```

3.3. Rekonstruksi Solusi

Serupa dengan algoritma pathfinding lainnya, setelah goal state berhasil dicapai, program melakukan penelusuran mundur (backtracking) menggunakan tabel parent dan parentMove. Urutan gerakan (L, R, U, D) kemudian dibalik posisinya (std::reverse) agar merepresentasikan langkah kronologis dari awal permainan hingga karakter mencapai titik kemenangan. Hasil rekonstruksi dan metrik performa (waktu eksekusi, jumlah iterasi) kemudian dibungkus ke dalam objek ui::SolutionLog.

4. A*

4.1. Fungsi Heuristik dan Cost Scaling (heuristic)

Fungsi heuristic pada A* dirancang untuk memberikan estimasi biaya yang bersifat admissible (tidak pernah melebihi biaya asli). Algoritma menghitung Manhattan Distance secara kumulatif, berawal dari posisi karakter saat ini, menyusuri setiap checkpoint angka yang belum dikunjungi sesuai urutan, hingga berakhir di titik keluar ('O'). Karena matriks permainan memiliki cost traversal untuk setiap tile, jarak langkah (distance) murni tidak sepadan dengan nilai biaya (cost) $g(n)$. Oleh karena itu, total jarak dikalikan dengan bobot tile paling minimum yang ada di peta (minWeight). Hal ini

memastikan bahwa estimasi heuristik ($h(n)$) selalu proporsional dan tidak pernah melakukan overestimate.

```
static int heuristic(const core::State& state, const core::Graph& graph) {
    int minWeight = graph.getMinWeight();
    const auto& checkpoints = graph.getCheckpointPositions();
    // ... [Inisialisasi jarak dan status kunjungan]

    // Kalkulasi jarak kumulatif antar checkpoint yang belum dikunjungi
    for (int i = 0; i < totalCp; i++){
        if (visited & (1 << i)) continue;
        totalDist += std::abs(current.row - checkpoints[i].row)
                    + std::abs(current.col - checkpoints[i].col);
        current = checkpoints[i];
    }
    // Tambahkan jarak dari checkpoint terakhir ke titik tujuan
    totalDist += std::abs(current.row - graph.getGoalPos().row) +
std::abs(current.col - graph.getGoalPos().col);

    return minWeight * totalDist; // Cost scaling agar admissible
}
```

4.2. Struktur Data Pendukung

Implementasi ini menggunakan beberapa struktur data penting untuk memastikan pencarian berjalan efisien:

1. **openList (Antrean Prioritas):** Digunakan untuk menyimpan AStarNode yang belum diekspansi. Antrean ini diurutkan menggunakan komparator khusus ACompare yang memastikan node dengan nilai $f(n)$ ($a.f > b.f$) paling kecil selalu berada di puncak antrean (min-heap).
2. **closedList (Set):** Menyimpan daftar state yang sudah diekspansi secara penuh untuk mencegah siklus atau evaluasi ulang pada node yang sama.
3. **gScore (Hash Map):** Bertugas melacak rekor biaya aktual $g(n)$ paling minimum yang pernah ditemukan untuk mencapai suatu state.

4.3. Siklus Ekspansi dan Relaksasi

Selama openList tidak kosong, algoritma mengambil node dengan evaluasi $f(n)$ terbaik. Di sini, program menerapkan dua lapis pruning:

1. Mengabaikan node dengan nilai $g(n)$ yang sudah usang (stale g) bila dibandingkan dengan catatan di gScore.
2. Mengabaikan node yang sudah terdaftar di closedList.

Jika state lolos pengecekan dan belum mencapai goal, state tersebut dimasukkan ke closedList dan dibangkitkan suksesornya.

Untuk setiap kemungkinan gerakan:

1. Dihitung kandidat biaya baru ($\text{possibleG} = \text{cur.g} + \text{succ.cost}$).
2. Algoritma memeriksa apakah state suksesor belum pernah dicatat di gScore, atau kandidat biaya possibleG ternyata lebih murah daripada rekor biaya sebelumnya.
3. Jika salah satu syarat di atas terpenuhi, proses relaksasi dilakukan: gScore, cameFrom, dan cameFromMove diperbarui; kemudian nilai $f(n)$ dihitung ($\text{possibleG} + \text{heuristic}()$) sebelum akhirnya suksesor dimasukkan ke dalam openList.

```
// ... [Pengecekan closedList dan perhitungan possibleG]
auto succG = gScore.find(succ.state);

// Relaksasi jika state baru atau ditemukan jalur yang lebih murah
if (succG == gScore.end() || possibleG < succG->second){
    gScore[succ.state] = possibleG;
    cameFrom[succ.state] = cur.state;
    cameFromMove[succ.state] = succ.move;

    // Hitung fungsi  $f(n) = g(n) + h(n)$ 
    int f = possibleG + heuristic(succ.state, graph);
    openList.push({succ.state, possibleG, f, succ.move});
}
```

4.4. Rekonstruksi Solusi

Apabila pencarian dihentikan karena goal state tercapai, program akan melakukan penelusuran balik (backtracking) dari titik keluar menuju posisi awal menggunakan tabel cameFrom dan cameFromMove. Vektor riwayat perjalanan kemudian dibalik (std::reverse) menjadi urutan kronologis. Hasil runutan gerakan, posisi state, dan metrik performa algoritma (waktu dan iterasi) dikemas dalam bentuk ui::SolutionLog untuk dilaporkan atau divisualisasikan.

BAB IV

PENGUJIAN

Tabel 4.1. Hasil Pengujian Algoritma

Kasus Uji	Hasil
<pre> 7 XXXXXX X0***X X**X**X X***OX X1***LX XZ**X*X XXXXXX 999 999 999 999 999 999 999 999 3 5 2 8 1 999 999 7 4 999 6 9 999 999 2 8 3 5 4 999 999 1 7 2 3 999 999 999 9 3 4 999 8 999 999 999 999 999 999 999 999 </pre>	<pre> === UCS === Algorithm: UCS Found: YES Steps: 8 Iterations: 13 Time: 0 ms Log saved: log_UCS_20260510_225236.txt === AStar === Algorithm: A* Found: YES Steps: 8 Iterations: 13 Time: 0 ms Log saved: log_A*_20260510_225236.txt === GBFS === Algorithm: GBFS Found: YES Steps: 8 Iterations: 11 Time: 0 ms Log saved: log_GBFS_20260510_225236.txt </pre>

```

7
XXXXXXX
XZ***0X
X*X*X*X
XLXX*X
XL***1X
X*X**OX
XXXXXXX
999 999 999 999 999 999 999
999 2 4 3 1 5 999
999 6 999 7 999 2 999
999 3 999 9 999 4 999
999 1 5 2 3 6 999
999 8 999 4 1 3 999
999 999 999 999 999 999 999

```

```

=== UCS ===
Algorithm: UCS
Found: YES
Steps: 2
Iterations: 4
Time: 0 ms
Log saved: log_UCS_20260510_225236.txt

```

```

=== AStar ===
Algorithm: A*
Found: YES
Steps: 2
Iterations: 3
Time: 0 ms
Log saved: log_A*_20260510_225236.txt

```

```

=== GBFS ===
Algorithm: GBFS
Found: YES
Steps: 2
Iterations: 3
Time: 0 ms
Log saved: log_GBFS_20260510_225236.txt

```

```

6 10
XXXXXXXXXX
XZ**X***0X
X*****X
X*XX***LX
X0***X*LX
XXXXXXXXXX
999 999 999 999 999 999 999 999 999
999 1 3 5 999 2 4 6 7 999
999 8 2 4 1 9 3 5 2 999
999 6 999 999 3 7 2 4 8 999
999 4 1 5 2 6 999 3 9 999
999 999 999 999 999 999 999 999 999

```

```

=== UCS ===
Algorithm: UCS
Found: YES
Steps: 7
Iterations: 12
Time: 0 ms
Log saved: log_UCS_20260510_225236.txt

```

```

=== AStar ===
Algorithm: A*
Found: YES
Steps: 7
Iterations: 12
Time: 0 ms
Log saved: log_A*_20260510_225236.txt

```

```

=== GBFS ===
Algorithm: GBFS
Found: YES
Steps: 7
Iterations: 12
Time: 0 ms
Log saved: log_GBFS_20260510_225236.txt

```



```

1 10
XXXXXXXXXX
XL*7X4**3X
XXX***XL*X
X0X6*5***X
X***X*L*X
X*XX81**2X
X***X***X
X*X***X
X***X*9X
XXZ*LL*0XX
XXXXXXXXXX
999 999 999 999 999 999 999 999 999 999
999 3 5 8 999 1 7 4 2 999
999 999 999 2 6 9 999 5 1 999
999 4 999 7 3 5 8 1 6 999
999 1 8 2 5 999 4 7 3 999
999 6 999 999 2 8 5 3 4 999
999 7 4 1 999 6 2 9 8 999
999 5 999 3 8 4 1 6 7 999
999 2 9 6 1 7 999 5 8 999
999 999 3 5 8 2 4 1 999 999
999 999 999 999 999 999 999 999 999

```

```

=== UCS ===
Algorithm: UCS
Found: YES
Steps: 19
Iterations: 65
Time: 0 ms
Log saved: log_UCS_20260510_225236.txt

```

```

=== AStar ===
Algorithm: A*
Found: YES
Steps: 19
Iterations: 60
Time: 0 ms
Log saved: log_A*_20260510_225236.txt

```

```

=== GBFS ===
Algorithm: GBFS
Found: YES
Steps: 19
Iterations: 46
Time: 0 ms
Log saved: log_GBFS_20260510_225236.txt

```

```

20 20
XXXXXXXXXXXXXXXXXXXX
XZ*****0X*****X
X*****X*****L**X
X*****L***X*****X
X*****X*****L**X
X*XL*****L***X
X*****X*****L***X
X*****X*****L***X
X*****X*****L***X
X*****L*****X
X*****L*****X
X**2*****3X**X
X**X*****L*****X
X*****L*****X
X*****L*****X
X*L*****L*****X
X*****L*****X
X*****L*****X
X*****L*****X
X*****L*****X
X*****L*****X
XXXXXXXXXXXXXXXXXXXX
999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999
999 2 1 5 4 4 3 2 9 999 2 7 1 1 2 4 4 9 1 999
999 9 4 9 7 4 8 5 1 999 3 7 6 5 3 4 6 2 2 999
999 7 2 6 6 999 5 1 8 999 9 2 7 2 9 5 6 4 2 999
999 1 4 5 2 4 2 7 5 999 8 6 3 6 6 4 5 2 3 999
999 9 999 4 3 8 7 5 9 4 6 1 4 1 6 7 5 2 4 999
999 6 4 8 7 8 3 5 999 3 4 9 9 5 7 7 6 4 3 999
999 9 8 2 1 2 3 3 7 2 7 999 8 9 5 9 1 2 999
999 9 5 6 2 5 7 3 8 1 5 9 3 9 2 5 9 4 3 999
999 6 3 9 9 1 6 8 1 2 6 5 4 1 4 2 2 8 2 999
999 9 3 3 8 9 3 5 9 7 4 9 4 5 7 6 999 8 9 999
999 8 2 999 4 4 2 6 1 9 4 4 1 2 1 4 2 1 6 999
999 2 9 4 5 8 4 9 3 8 4 8 7 4 2 2 7 6 7 999
999 7 8 1 2 1 7 6 999 2 4 4 4 9 8 3 7 3 5 999
999 8 4 2 8 9 2 1 9 1 2 4 3 7 8 8 4 7 1 999
999 3 7 1 7 5 8 5 7 9 8 3 4 5 4 1 9 1 999 999
999 6 1 1 8 9 9 3 1 9 2 3 2 2 4 999 7 2 4 999
999 1 2 7 9 6 5 4 6 4 5 7 3 5 8 6 2 1 8 999
999 2 2 9 4 9 5 3 6 2 4 6 5 3 8 9 5 9 1 999
999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999

```

```

=== UCS ===
Algorithm: UCS
Found: YES
Steps: 9
Iterations: 25
Time: 0 ms
Log saved: log_UCS_20260510_225236.txt

```

```

=== AStar ===
Algorithm: A*
Found: YES
Steps: 9
Iterations: 23
Time: 0 ms
Log saved: log_A*_20260510_225236.txt

```

```

=== GBFS ===
Algorithm: GBFS
Found: YES
Steps: 9
Iterations: 14
Time: 0 ms
Log saved: log_GBFS_20260510_225236.txt

```

<pre> 5 5 XXXXX X***X X*X*X XZ*OX XXXXX 999 999 999 999 999 999 2 3 4 999 999 1 999 2 999 999 5 6 3 999 999 999 999 999 999 </pre>	<pre> === UCS === Algorithm: UCS Found: YES Steps: 1 Iterations: 3 Time: 0 ms Log saved: log_UCS_20260510_225236.txt === AStar === Algorithm: A* Found: YES Steps: 1 Iterations: 3 Time: 0 ms Log saved: log_A*_20260510_225236.txt === GBFS === Algorithm: GBFS Found: YES Steps: 1 Iterations: 2 Time: 0 ms Log saved: log_GBFS_20260510_225236.txt </pre>
<pre> 5 7 XXXXXXX XZ*X*OX X***X*X X*XXXXX XXXXXXX 999 999 999 999 999 999 999 999 1 2 999 3 4 999 999 5 6 7 999 8 999 999 9 1 2 999 999 999 999 999 999 999 999 999 999 </pre>	<pre> === UCS === Algorithm: UCS Found: NO Steps: 0 Iterations: 6 Time: 0 ms Log saved: log_UCS_20260510_225236.txt === AStar === Algorithm: A* Found: NO Steps: 0 Iterations: 6 Time: 0 ms Log saved: log_A*_20260510_225236.txt === GBFS === Algorithm: GBFS Found: NO Steps: 0 Iterations: 6 Time: 0 ms Log saved: log_GBFS_20260510_225236.txt </pre>

Tabel 4.2. Hasil Render GUI

```

7
XXXXXXX
X0***X
X**X**X
X***OX
X1***LX
XZ**X*X
XXXXXXX
999 999 999 999 999 999 999
999 3 5 2 8 1 999
999 7 4 999 6 9 999
999 2 8 3 5 4 999
999 1 7 2 3 999 999
999 9 3 4 999 8 999
999 999 999 999 999 999 999

```

X	X	X	X	X	X	X
X	0	5	2	8	1	X
X	7	4	X	6	9	X
X	2	8	3	5	O	X
X	1	7	2	3	L	X
X	Z	3	4	X	8	X
X	X	X	X	X	X	X

```

7
XXXXXXX
XZ***OX
X*X*X*X
XLXX*X
XL***1X
X*X**OX
XXXXXXX
999 999 999 999 999 999 999
999 2 4 3 1 5 999
999 6 999 7 999 2 999
999 3 999 9 999 4 999
999 1 5 2 3 6 999
999 8 999 4 1 3 999
999 999 999 999 999 999 999

```

X	X	X	X	X	X	X
X	Z	4	3	1	0	X
X	6	X	7	X	2	X
X	L	X	X	X	4	X
X	L	5	2	3	1	X
X	8	X	4	1	O	X
X	X	X	X	X	X	X

```

6 10
XXXXXXXXXX
XZ**X***OX
X*****X
X*XX****LX
X0***X*LX
XXXXXXXXXX
999 999 999 999 999 999 999 999 999
999 1 3 5 999 2 4 6 7 999
999 8 2 4 1 9 3 5 2 999
999 6 999 999 3 7 2 4 8 999
999 4 1 5 2 6 999 3 9 999
999 999 999 999 999 999 999 999 999

```

X	X	X	X	X	X	X	X	X
X	Z	3	5	X	2	4	6	0
X	8	2	4	1	9	3	5	2
X	6	X	X	3	7	2	4	L
X	O	1	5	2	6	X	3	L
X	X	X	X	X	X	X	X	X

```

1 10
XXXXXXXXXX
XL*7X4**3X
XXX***XL*X
X0X6*5***X
X***X*L*X
X*XX81**2X
X***X***X
X*X*****X
X*****X*GX
XXZ*LL*0XX
XXXXXXXXXX
999 999 999 999 999 999 999 999 999 999
999 3 5 8 999 1 7 4 2 999
999 999 999 2 6 9 999 5 1 999
999 4 999 7 3 5 8 1 6 999
999 1 8 2 5 999 4 7 3 999
999 6 999 999 2 8 5 3 4 999
999 7 4 1 999 6 2 9 8 999
999 5 999 3 8 4 1 6 7 999
999 2 9 6 1 7 999 5 8 999
999 999 3 5 8 2 4 1 999 999
999 999 999 999 999 999 999 999 999 999

```

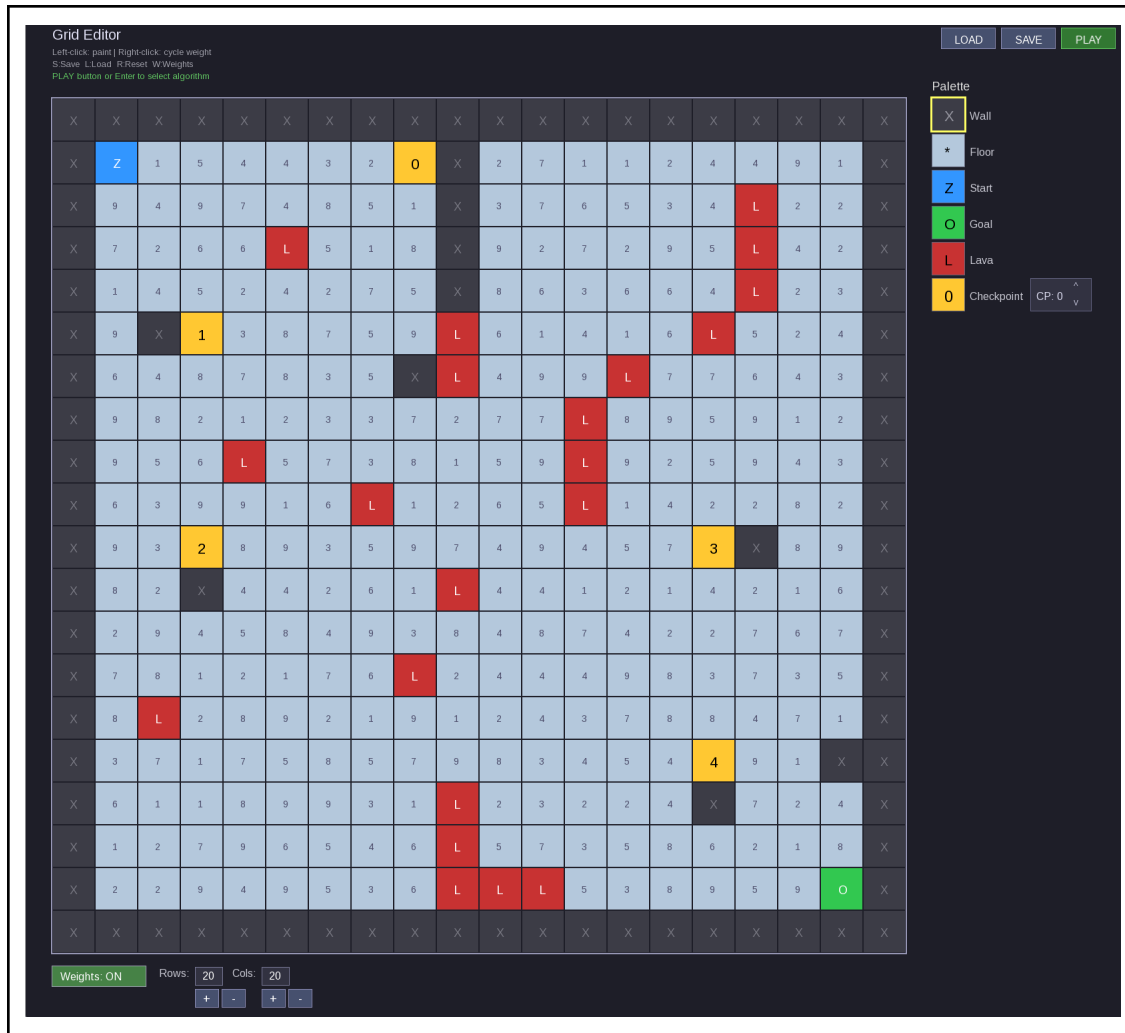
X	X	X	X	X	X	X	X	X	X
X	L	5	7	X	4	7	4	3	X
X	X	X	2	6	9	X	L	1	X
X	0	X	6	3	5	8	1	6	X
X	1	8	2	5	X	4	L	3	X
X	6	X	X	8	1	5	3	2	X
X	7	4	1	X	6	2	9	8	X
X	5	X	3	8	4	1	6	7	X
X	2	9	6	1	7	X	5	9	X
X	X	Z	5	L	L	4	O	X	X
X	X	X	X	X	X	X	X	X	X

```

20 20
XXXXXXXXXXXXXXXXXXXX
XZ*****0X*****X
X*****X*****L**X
X*****L*****L**X
X*****X*****L**X
X**X1*****L*****X
X*****X*****L*****X
X*****L*****L*****X
X***L*****L*****X
X*****L*****L*****X
X**2*****3X**X
X**X*****L*****X
X*****L*****X
X*****L*****X
X*L*****X
X*****4**X
X*****L*****X
X*****L*****X
X*****L*****X
XXXXXXXXXXXXXXXXXXXX
999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999
999 2 1 5 4 4 3 2 9 999 2 7 1 1 2 4 4 9 1 999
999 9 4 9 7 4 8 5 1 999 3 7 6 5 3 4 6 2 2 999
999 7 2 6 6 999 5 1 8 999 9 2 7 2 9 5 6 4 2 999
999 1 4 5 2 4 2 7 5 999 8 6 3 6 6 4 5 2 3 999
999 9 999 4 3 8 7 5 9 4 6 1 4 1 6 7 5 2 4 999
999 6 4 8 7 8 3 5 999 3 4 9 9 5 7 7 6 4 3 999
999 9 8 2 1 2 3 3 7 2 7 7 999 8 9 5 9 1 2 999
999 9 5 6 2 5 7 3 8 1 5 9 3 9 2 5 9 4 3 999
999 6 3 9 9 1 6 8 1 2 6 5 4 1 4 2 2 8 2 999
999 9 3 3 8 9 3 5 9 7 4 9 4 5 7 6 999 8 9 999
999 8 2 999 4 4 2 6 1 9 4 4 1 2 1 4 2 1 6 999
999 2 9 4 5 8 4 9 3 8 4 8 7 4 2 2 7 6 7 999
999 7 8 1 2 1 7 6 999 2 4 4 4 9 8 3 7 3 5 999
999 8 4 2 8 9 2 1 9 1 2 4 3 7 8 8 4 7 1 999
999 3 7 1 7 5 8 5 7 9 8 3 4 5 4 1 9 1 999 999
999 6 1 1 8 9 9 3 1 9 2 3 2 2 4 999 7 2 4 999
999 1 2 7 9 6 5 4 6 4 5 7 3 5 8 6 2 1 8 999
999 2 2 9 4 9 5 3 6 2 4 6 5 3 8 9 5 9 1 999
999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999

```

X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X
X	Z	1	5	4	4	3	2	0	X	7	1	1	1	2	4	4	9	1	X
X	9	4	9	7	4	8	5	1	X	9	7	6	5	3	4	L	2	2	X
X	7	2	6	4	L	5	L	8	X	6	2	7	3	9	6	L	4	2	X
X	1	4	5	2	4	2	7	5	X	6	3	5	6	4	L	2	2	X	X
X	9	X	1	3	8	7	5	9	L	6	1	4	1	6	L	5	2	4	X
X	6	6	8	7	8	3	2	7	L	4	5	9	L	7	7	6	4	2	X
X	9	8	2	1	2	3	3	7	2	7	7	L	6	9	5	9	1	2	X
X	9	5	6	L	1	7	9	8	1	5	9	L	1	2	5	9	4	3	X
X	8	3	9	8	1	6	L	1	2	9	5	L	1	4	2	2	8	2	X
X	9	1	2	8	9	3	5	9	7	4	9	4	5	7	3	X	8	9	X
X	6	7	X	4	8	2	6	1	L	6	4	1	2	1	4	2	1	5	X
X	2	9	4	5	8	4	3	2	8	4	8	7	4	2	2	7	6	7	X
X	7	8	1	2	1	7	6	L	2	4	4	4	5	8	5	7	3	5	X
X	8	L	2	8	9	2	1	9	1	2	4	3	7	6	9	6	7	1	X
X	3	7	1	7	5	8	5	7	9	9	3	4	5	4	4	9	1	5	X
X	6	1	1	8	5	9	3	1	L	9	3	2	2	4	X	7	2	X	X
X	1	2	7	9	6	5	4	5	L	5	7	3	5	8	8	2	1	9	X
X	2	2	9	4	9	5	2	6	L	L	5	3	8	9	5	9	O	X	X
X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X



Gambar 4.1.1 Grid Editor

Referensi

Works Cited

“A* Star Algorithm Open and Closed Lists.” *Stack Overflow*,
stackoverflow.com/questions/45577114/a-star-algorithm-open-and-closed-lists.

Aho, Alfred V, et al. *Compilers : Principles, Techniques, and Tools*. Noida, India, Dorling
Kindersley (India) Pvt. Ltd., Licensees Of Pearson Education In South Asia, 2014.

“Compiler Design - Lexical Analysis - Tutorialspoint.” *Tutorialspoint.com*, 2020,
www.tutorialspoint.com/compiler_design/compiler_design_lexical_analysis.htm.

Huddar, Mahesh. “Greedy Best First Search Algorithm Solved Example in Artificial
Intelligence.” *Youtu.be*, 2026, youtu.be/GVvN0ikNekw?si=ilQLkbSM8wV3JU32.

Huddar, Mahesh . “Uniform Cost Search Algorithm | UCS Search Algorithm in Artificial
Intelligence by Mahesh Huddar.” *Youtu.be*, 2026,
youtu.be/8ofimg8cnRE?si=79ItuhBxD9LQfUKk.

“Implementation of A*.” *Www.redblobgames.com*,
www.redblobgames.com/pathfinding/a-star/implementation.html.

Lampiran

Link Github: https://github.com/aufafaza/Tucil3_13524027_13524077

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



Aufa Rienaldifaza Ahmad



Yavie Azka Putra Araly

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	✓	
4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	✓	
5	Program memiliki Graphical User Interface (GUI)	✓	
6	Ada algoritma <i>pathfinding</i> alternatif selain dari spesifikasi wajib (Algoritma X dan Algoritma Y)		✓
7	Ada tambahkan dua alternatif heuristik selain yang		✓

	utama (Heuristik X dan Heuristik Y)		
--	-------------------------------------	--	--